Ensino que te conecta

BACHARELADO EM

ENGENHARIA DE SOFTWARE

ACCELERATION OF PATTERN MATCHING ALGORITHMS USING PARALLEL PROGRAMMING

MATHEUS A. C. BARROS

Brasília - DF, 2025

MATHEUS A. C. BARROS

ACCELERATION OF PATTERN MATCHING ALGORITHMS USING PARALLEL PROGRAMMING

Qualification Exam document submitted as a partial requirement for the Bachelor's Degree in Software Engineering at the Instituto Brasileiro de Ensino, Desenvolvimento e Pesquisa (IDP).

Orientador
Jeremias M. Gomes

Brasília - DF, 2025

MATHEUS A. C. BARROS

ACCELERATION OF PATTERN MATCHING ALGORITHMS USING PARALLEL PROGRAMMING

Qualification Exam document submitted as a partial requirement for the Bachelor's Degree in Software Engineering at the Instituto Brasileiro de Ensino, Desenvolvimento e Pesquisa (IDP).

Aprovado em 08/05/2025

Banca Examinadora

Patrícia Oliveira

Lucas Maurício

Jeremias M. Gomes

ABSTRACT

Keywords: Aho–Corasick, Parallel Programming, Parallel Computing, Multi-pattern Matching, YARA, Malware Detection, POSIX Threads, Intrusion Detection Systems .

LIST OF FIGURES

1 Ideal vs. Amdahl's theoretical speedup ($s = 0.1$). 9

2 Example trie storing {"car", "cat", "cart", "dog"}. Nodes marking the end of a complete word are shown with a double circle. 11

3 Flowchart of the Aho–Corasick matching phase. Start at the root with $i = 1$, attempt each goto($\textit{current_state}, a_i$), follow failure links on a "no" decision, report matches via output($\textit{current_state}$) on a "yes," advance i, and terminate when $i > n$..... 17

LIST OF TABLES

1 Cronograma de Atividades do TCC (Fev-Jul) 25
2 Cronograma de Atividades do TCC (Ago-Dez) 26

CONTENTS

1 Introduction 2

1.1 Motivation 3

1.2 Objectives 3

1.3 Structure of the Thesis 4

2 Theoretical Framework 6

2.1 Fundamentals of Parallelism 6

2.1.1 Parallel Computing Architectures 6

2.1.2 Parallel Programming With Threads 7

2.1.3 Parallel Performance Evaluation Metrics 8

2.2 Tries (Prefix Trees) 10

2.2.1 Definition and Basic Structure 10

2.2.2 Core Operations 11

2.2.3 Advantages and Performance Characteristics 12

2.2.4 Space Complexity Considerations 12

2.2.5 Foundational Role in String Processing 12

2.3 Pattern Matching 13

2.3.1 The Pattern Matching Problem 13

2.3.2 Fundamental Algorithms for Exact Single-Pattern Matching 13

2.3.3 Limitations of Iterating Single-Pattern Algorithms 14

2.3.4 The Need for Specialized Multi-Pattern Algorithms 15

2.4 The Aho-Corasick Algorithm 15

2.4.1 Context 15

2.4.2 Algorithm Components and Construction 16

2.4.3 Matching Phase 16

2.4.4 Time and Space Complexity 17

2.4.5 Applications and Relevance 18

3 Systematic Review 20

3.1 Research Questions 20

3.2 Search Strategy and Study Selection Protocol 20

3.2.1 Data Sources 20

3.2.2 Search Queries 21

3.2.3 Inclusion (IC) and Exclusion (EC) Criteria 21

3.2.4Selection Process and Initial Search Outcomes *22*

4 Proposal **25**

4.1 Cronograma 25

References **27**

1

1 INTRODUCTION

Advancements in semiconductor technology have historically driven computational power growth. For decades, the performance of single-core processors followed the trend predicted by *Moore's Law*, which forecast the doubling of transistors on a chip every eighteen months [1]. However, in the mid-2000s, the hardware industry encountered significant physical barriers, such as excessive power consumption and challenges in heat dissipation, which limited the increase in processor clock frequency. The response to these technological limitations was a change in processor design: instead of focusing on making a single core faster, the emphasis shifted to integrating multiple processing cores onto a single chip. Thus, multi-core processors emerged as the dominant architecture in modern computers. This transition opened up new possibilities for performance enhancement; however, to take advantage of the performance gains offered by multi-core processors, programs must be intentionally developed or adapted to execute tasks in parallel.

This architectural shift from sequential to parallel execution profoundly impacts high-performance applications. In information security, tools for malware analysis and network intrusion detection must process vast data volumes rapidly to identify malicious patterns and enable prompt responses. YARA, an industry standard for pattern-based malware classification, exemplifies this challenge. It utilizes the efficient Aho-Corasick algorithm as the core of its multi-pattern matching engine, allowing it to search thousands of string patterns simultaneously [2].

Despite its efficiency for multi-pattern searching, YARA's internal implementation of the Aho-Corasick algorithm is predominantly sequential. While YARA supports *inter-file parallelism*—concurrently scanning multiple smaller files across different threads—it lacks *intra-file parallelism*. This means that when scanning a single, large target (e.g., a full disk image or a large network packet capture), the entire computational load falls upon a single CPU core. This limitation constitutes a significant performance bottleneck in critical, real-world security scenarios, where rapid analysis of large-volume targets is paramount. As the size of data in security incidents continues to grow, this bottleneck becomes increasingly prohibitive.

1.1 MOTIVATION

The identified performance limitation of YARA's sequential engine when scanning large files presents a clear opportunity for optimization through parallel computing. Prior literature demonstrate that parallelizing the Aho-Corasick algorithm is both viable and effective, with significant speedups reported in domains such as bioinformatics [3]. However, a systematic review of the literature reveals that direct implementation, integration, and empirical evaluation of a parallel Aho-Corasick engine within the YARA framework remains an unexplored research area.

Given the ever-increasing volume of data to be scanned and the growing demand for efficient malware detection, improving the performance of pattern-matching engines remains a relevant and pressing challenge. While existing solutions offer powerful detection capabilities, scaling them to handle large workloads efficiently on modern multi-core systems is still a non-trivial problem. Addressing this challenge is crucial for ensuring that tools like YARA remain effective and practical in real-world scenarios where speed and accuracy are essential."

1.2 OBJECTIVES

The general objective of this thesis is to demonstrate that the performance of YARA's pattern matching engine can be significantly accelerated for large-file scanning scenarios by introducing *intra-file parallelism* on multi-core CPU architectures.

To achieve this general objective, the following specific objectives are defined:

1. To design and implement a parallel version of the Aho-Corasick.
2. To conduct a rigorous experimental evaluation to quantify the performance gains of the parallel implementation in terms of speedup and throughput.
3. To analyze the scalability of the proposed solution under different workload and configuration scenarios.

1.3 STRUCTURE OF THE THESIS

This paper is organized as follows. Chapter 2 presents the Theoretical Framework, introducing the fundamentals of parallel computing, including architectures, performance metrics, and thread-based programming models. It also discusses the core data structures and algorithms relevant to this work, particularly Tries, the general problem of pattern matching, and a detailed analysis of the Aho-Corasick algorithm. Chapter 3 describes the Systematic Literature Review, presenting the research questions, search strategy, and selection criteria adopted to identify the state of the art and define the research gap. Chapter 4 outlines the Proposed Solution, presenting the planned parallel scanning strategy and the main design considerations that will guide the development of this work.

2

2 THEORETICAL FRAMEWORK

2.1 FUNDAMENTALS OF PARALLELISM

Parallelism in computing refers to the simultaneous execution of multiple tasks or parts of the same task, with the primary goal of reducing a program's total execution time. It is crucial to distinguish parallelism from concurrency. Concurrency occurs when multiple tasks are in progress at a given time, but may not be executing simultaneously. A single-core system can exhibit concurrency by rapidly switching between the execution of different tasks, simulating parallelism through preemption. True parallelism, on the other hand, requires hardware with multiple processing units [4].

2.1.1 Parallel Computing Architectures

Parallel computer architectures are classified according to various criteria. A prominent taxonomy, proposed by Flynn, categorizes these architectures based on instruction and data streams [5]:

- **SISD (Single Instruction, Single Data)**: Represents a conventional single-processor architecture, where a single instruction operates on a single data stream.
- **SIMD (Single Instruction, Multiple Data)**: Involves multiple processing units that execute the same instruction concurrently on different data elements.
- **MISD (Multiple Instruction, Single Data)**: Characterizes systems where multiple processing units apply distinct instructions on the same data stream.
- **MIMD (Multiple Instruction, Multiple Data)**: Involves multiple processing units that execute distinct instructions on distinct data streams independently. This paradigm is directly related to multi-core architectures.

Within the MIMD category, systems are further distinguished based on their memory organization [1]:

- **Shared-Memory Systems**: In these systems, multiple processors share a physical memory address space. Inter-processor communication (IPC) is typically achieved by reading from and writing to shared variables in memory or through

message passing mechanisms. Multi-core processors are the most common and widely available example of this architecture, where all cores on a chip share access to main memory.

- **Distributed-Memory Systems**: Each processor in these systems possesses their own locally accessible memory, and communication between processors is usually done by exchanging messages across an interconnection network. Computer clusters are examples of this architectural model.

The present work focuses on the acceleration of algorithms in environments with multi-core processors, which fall into the shared memory MIMD category. A multi-core processor (MCP) is a single chip that contains two or more central processing units (CPUs) called 'cores'. Each of these cores is capable of reading and executing program instructions independently, essentially functioning as a complete processor.

2.1.2 Parallel Programming With Threads

Within shared-memory systems, such as multi-core processors, software-level parallelism is commonly achieved through the execution of multiple threads within a single process.

Processes vs. Threads

A process is an instance of an executing program, an entity possessing its own dedicated virtual address space, file descriptors, environment variables, and other Operating System (OS) resources, all isolated from other processes [1].

A thread, on the other hand, is a lighter unit of execution started by a process. The same process can control, concurrently or in parallel, multiple threads. A key characteristic is that all threads within a process share its virtual address space, which encompasses executable code, global data, and the heap segment. However, each thread maintains its private resources, which are essential for its independent execution context:

- A unique identifier (thread ID).
- A set of CPU registers, including the program counter (PC).
- A separate execution stack, used for local variables and function call control.
- Thread-specific state, such as signal masking and scheduling priority.

POSIX Thread Model (Pthreads)

The POSIX Thread Model, commonly referred to as "Pthreads", is a standard that defines an application programming interface (API) for thread creation and management, as specified by IEEE Std 1003.1c-1995. Pthreads provides a portable suite of data types, functions, and constants that allows developers to create multithreaded

applications that are compilable and executable across diverse POSIX-compliant operating systems (e.g., Linux, macOS, and various Unix variants). The Pthreads API provides functionality for managing the thread lifecycle—including creation, termination, and joining—and, critically, incorporates synchronization primitives. These primitives, such as mutexes and condition variables, are indispensable for orchestrating access to shared resources within shared-memory environments [1, 4].

2.1.3 Parallel Performance Evaluation Metrics

Performance evaluation is fundamental to understanding the effectiveness of a parallel program. The following metrics compare the execution time of the parallel program with its equivalent sequential counterpart [6].

Execution Time and Overhead

The performance of a program is measured by its execution time. For parallel programs, the following terms are defined:

- **Ts**: The execution time of a baseline sequential algorithm executed on a single processor unit.
- **Tp**: The execution time of the parallel program utilizing p processors.

Ideally, `Tp` is expected to be substantially less than `Ts`. However, parallel execution introduces *overhead*. This encompasses the time spent by processors on tasks that do not directly contribute to problem resolution, including communication, synchronization, load balance, and other computations inherent to the parallel implementation.

Speedup

Speedup (`S`) measures the performance gain achieved through parallel execution relative to sequential execution and it is referenced as Equation 1. It is defined as the ratio of sequential execution time to parallel execution time:

$$S = \frac{T_s}{T_p} \tag{1}$$

An ideal, or linear, speedup would be `S = p`. Amdahl's Law, establishes a theoretical upper bound on the speedup achievable by parallelizing a program with a fixed problem size. `s` represents the fraction of a program's sequential execution time that is inherently serial (non-parallelizable), and `f` is the parallelizable fraction (`s + f = 1`), the maximum speedup on `p` processors is given by:

$$S \leq \frac{1}{s + f/p} \tag{2}$$

In the limit, as the number of processors (`p`) tends to infinity, the maximum speedup is bounded by $\frac{1}{s}$. This implies that even a small sequential fraction (`s` large) can severely

limit the total speedup, regardless of the number of processors available. Figure 1 illustrates the theoretical speedup according to Amdahl's Law compared to ideal speedup for a given sequential fraction. A complementary perspective is offered by the Gustafson-Barsis Law. This law argues that a speedup approximately linear

$$S_{\text{escalado}} = s + (1 - s) \cdot p \tag{3}$$

is achievable for problems that benefit from this scalability. This formulation suggests that for problems that can be scaled, it is possible to obtain speedup approximately linear with the number of processors, even in the presence of a serial fraction. Both Amdahl's and Gustafson's laws are important for understanding the limits and potential of parallelism in different scenarios.

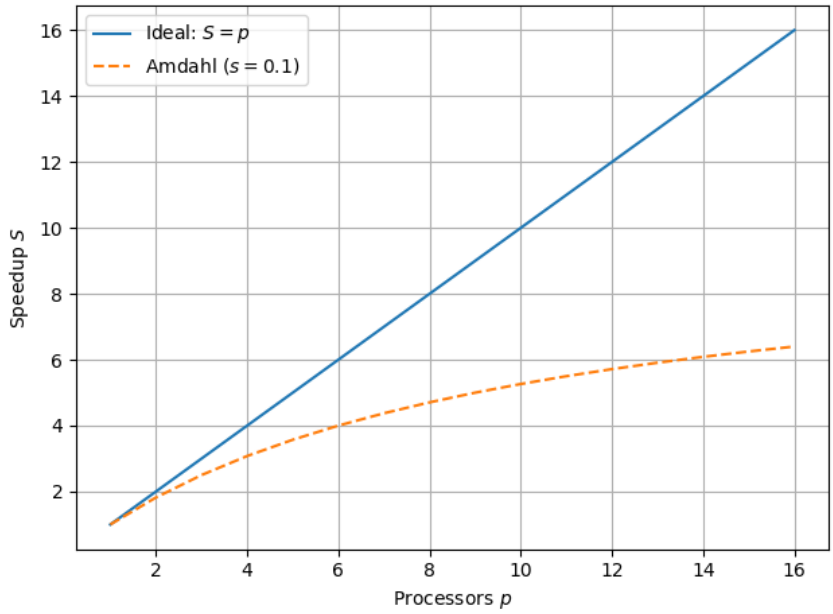

Figure 1: Ideal vs. Amdahl's theoretical speedup ($s = 0.1$).

Efficiency

Efficiency (E) measures the effectiveness of processor utilization in a parallel program. It is defined as the ratio of speedup to the number of processors p:

$$E = \frac{S}{p} = \frac{T_s}{p \cdot T_p} \tag{4}$$

Efficiency values range between 0 and 1 (inclusive), often expressed as a percentage (0% to 100%). An efficiency of 1 (or 100%) signifies ideal, linear speedup, implying that all processors are contributing productively throughout the execution. In practice, efficiency typically diminishes as the processor count increases, primarily due to the escalating impact of parallel overhead.

Scalability

In parallel computing, scalability denotes a system's ability to sustain high performance (often via efficiency) as both the number of processors and/or the problem size grow.

- **Strong Scalability**: Measures how execution time changes when you increase the processor count p for a fixed total problem size, denoted W (for example, the total number of array elements to process). The goal is to keep efficiency nearly constant as p increases. Amdahl's Law directly applies in this scenario.
- **Weak Scalability**: Measures how execution time changes when you increase both the processor count p and the total problem size W such that the work per processor (W/p) stays constant. The aim is to maintain efficiency as p and W grow proportionally. Gustafson's Law is most relevant here.

A parallel system is considered scalable if its efficiency remains above an acceptable threshold as both the number of processors p and the problem size W increase.

2.2 TRIES (PREFIX TREES)

This section elucidates the Trie data structure, detailing its defining properties, core operational mechanics, performance aspects, and its significance as a foundational element in advanced string processing algorithms, particularly the Aho-Corasick algorithm.

2.2.1 Definition and Basic Structure

A Trie—also known as a prefix tree—is a specialized tree-like data structure optimized for the efficient storage and retrieval of a dynamic set of strings. Edward Fredkin introduced the term "trie" in 1960, highlighting its primary application in information retrieval [7, 8]. Figure 2 illustrates an example of a trie storing a set of words.

Tries store strings by organizing nodes so that each path from the root to a node represents a unique prefix. In a standard R-way trie, each node can branch to R children, where R denotes the size of the character alphabet. The existence of a parent-child edge highlights two key aspects of tries:

- The edge's position (index) within the parent node's array of R possible child links directly corresponds to a unique character in the alphabet. For instance, index 0 might correspond to 'a', index 1 to 'b', and so on, according to a predefined mapping.
- The character associated with this edge extends the prefix represented by the parent; this implies that at least one word (or key) in the trie contains this character sequence.

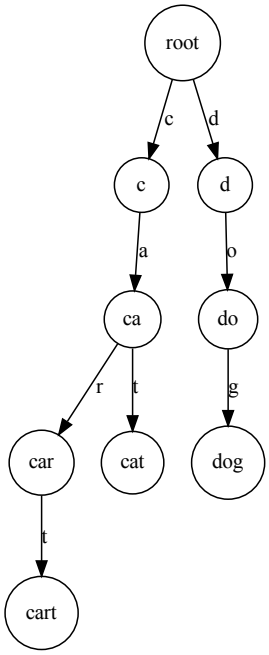

Figure 2: Example trie storing {“car”, “cat”, “cart”, “dog”}. Nodes marking the end of a complete word are shown with a double circle.

To signify that such a prefix also constitutes a complete key present in the trie, the node reached at the end of this path is typically marked by storing an associated non-null value or a boolean flag. The characters themselves are implicitly encoded by the trie’s structure rather than being explicitly stored within each node.

2.2.2 Core Operations

Tries support several primary operations, including search, insertion, and deletion [9].

Search: Locating a key involves traversing the trie starting from the root, following the successive characters of the target key. Each character dictates which child link to follow. The search operation succeeds if a complete path corresponding to all characters of the key exists and the node reached at the end of this path is appropriately marked as representing a complete key. The search fails if, at any point, a required link is null (i.e., no path for the current character exists) or if the path is fully traversed but the final node is not marked as a complete key.

Insertion: The insertion process begins similarly to a search, by traversing the trie according to the characters of the key to be added. If the path for the key does not extend to its full length (i.e., a null link is encountered), new nodes are created and linked to form the remainder of the path. The node corresponding to the final character of the inserted key is then marked to indicate that it is a complete key in the set.

Deletion: Deleting a key first involves locating the node corresponding to the key's final character and removing its end-of-word marker. If there are no child links (meaning it is not a prefix for any other key), it can be deleted. This deletion may proceed recursively: when a child node is removed, if its parent has no other children and the parent node is not marking a key, the parent node becomes redundant and can be deleted. This process continues up to the root as long as parent nodes become redundant.

2.2.3 Advantages and Performance Characteristics

Tries provide distinct advantages for operations on string collections.

Time Complexity: A primary advantage is that the time required for search and insertion operations depends primarily on the length of the key (L), not on the total number (N) of keys stored. Specifically, these operations typically require examining a number of nodes proportional to L. Sedgewick & Wayne (2011) state that the number of array accesses (node visits) is at most $L + 1$ [9]. This $O(L)$ performance makes tries highly efficient for prefix-based searches (e.g., retrieving all keys starting with a given prefix) and ensures that search performance does not substantially degrade as the dataset size increases.

Prefix Sharing: When stored strings share common prefixes, tries inherently exploit this by having those strings share the initial path from the root. This sharing can lead to significant space savings compared to storing each string independently, particularly if there is substantial prefix overlap among keys [7].

2.2.4 Space Complexity Considerations

While prefix sharing can save space, the overall space complexity of R-way tries requires careful consideration. Each node in an R-way trie conceptually provides R potential links. If the alphabet size R is large (e.g., for Unicode) and the actual branching factor at most nodes is low (a sparse trie), allocating space for all R links per node can lead to considerable memory consumption due to numerous unused (null) links [8, 9].

2.2.5 Foundational Role in String Processing

The trie data structure is a cornerstone for numerous advanced string processing algorithms. Critically for this work, the Aho-Corasick algorithm, engineered for the efficient, simultaneous matching of multiple patterns within a text, employs a trie—constructed from the set of patterns (the dictionary)—as its initial structural framework [10]. A thorough grasp of trie principles is thus indispensable for developing and analyzing the Aho-Corasick algorithm.

2.3 PATTERN MATCHING

This section introduces the fundamental problem of pattern matching within strings, explores key algorithmic approaches to its solution, and underscores its significance across various computational domains. This foundational discussion provides the essential context for understanding advanced multi-pattern matching algorithms, such as the Aho-Corasick algorithm. In general information processing, pattern (or *string*) matching underpins the functionality of text editors and web search engines for keyword searches. The field of computational biology extensively utilizes pattern matching for analyzing genetic sequences [9]. Furthermore, in the field of information security, pattern matching is crucial: Network Intrusion Detection Systems (NIDS) utilize it to identify malicious signatures in network traffic. Tools like YARA, for instance, define rules based on textual or binary patterns to classify and identify malware [2]. Later in this work, we'll explore the role of pattern matching for NIDS in greater detail.

2.3.1 The Pattern Matching Problem

The *string matching* problem consists in determining whether a string P (the *pattern*) of length M occurs within a string T (the *text*) of length N, where $M \leq N$. Formally, it seeks all positions pos such that the following condition holds for every $i \in \{0, 1, \ldots, M-1\}$:

$$P[i] = T[pos + i], \quad \text{with} \quad pos + i < N \tag{5}$$

Each valid pos corresponds to an occurrence of the pattern in the text. Beyond simply finding the first match, the problem often extends to locating all occurrences, enumerating them, or extracting surrounding contextual information for each match [9].

2.3.2 Fundamental Algorithms for Exact Single-Pattern Matching

Pattern matching algorithms are generally categorized based on the number of patterns they process and the exactness of the match. This thesis focuses on *Exact Matching*, where the pattern must correspond precisely to a segment of the text. A key distinction for this study lies between algorithms designed for *Single-Pattern Matching* and those optimized for *Multi-Pattern Matching*, where the objective is to identify all occurrences of any pattern from a predetermined set of patterns within a given text [10, 11].

The Naive Algorithm

The naïve or brute-force algorithm represents the most straightforward method for exact pattern matching. Its operation involves iteratively aligning the pattern P with the text T at every possible starting position i (from 0 to $N - M$). For each alignment, P is compared character by character, from left to right, against the corresponding substring. If all characters match, an occurrence is reported [11].

Following this comparison (whether a match or a mismatch), the algorithm shifts P

one position to the right relative to T and proceeds to the next alignment. Despite its simplicity, Knuth, Morris, and Pratt (1977) state that this method can be highly inefficient [12]. Its worst-case time complexity is $O(M \cdot N)$ character comparisons. This occurs, for example, when searching for a pattern like $P = a^n b$ within a text $T = a^{2n} b$, where many partial matches occur. To address these limitations, more advanced pattern matching algorithms include a preprocessing phase for the pattern P. This phase extracts structural information about P that is then utilized during the search, enabling larger shifts of the pattern relative to the text and thus reducing the total number of character comparisons.

Knuth-Morris-Pratt (KMP) Algorithm

The KMP algorithm [12] efficiently resolves the pattern matching problem with an optimal worst-case time complexity of $O(N + M)$. Unlike the naïve approach, KMP avoids re-checking characters that have already been matched. To achieve this, KMP first preprocesses the pattern P by constructing a special array known as the *failure array* (or *prefix array*), typically denoted by $p[1..M]$. This array indicates the length of the longest prefix that is also a suffix for each prefix of P. With this information, when a mismatch occurs during the search, the algorithm utilizes the failure array to determine how far the pattern should shift to the right, bypassing unnecessary comparisons and preventing redundant checks.

Boyer-Moore (BM) Algorithm

The Boyer-Moore (BM) algorithm [13] is widely recognized for its excellent practical performance, often outperforming linear-time methods in typical cases. Unlike the KMP algorithm, Boyer-Moore typically compares characters starting from the end of the pattern and moves backward. Its efficiency primarily stems from two rules (or heuristics):

- **Bad Character Rule**: When a mismatch occurs, this rule uses the character from the text that caused the mismatch to determine how far the pattern can safely shift. The algorithm shifts the pattern to align this mismatched text character with its last occurrence in the pattern or moves completely past the mismatched character if it does not appear in the pattern.
- **Good Suffix Rule**: When a mismatch happens after matching one or more characters from the end of the pattern, this rule uses the matched suffix to decide how far to shift the pattern. It searches for another occurrence of the matched suffix within the pattern, ensuring that no previously matched characters are unnecessarily compared again.

These two rules combined enable the Boyer-Moore algorithm to skip larger portions of the text, significantly reducing the total number of comparisons.

2.3.3 Limitations of Iterating Single-Pattern Algorithms

When the task involves searching for multiple patterns simultaneously from a set $K = \{P_1, P_2, \ldots, P_k\}$ within a text T, simply applying a single-pattern matching algorithm for each P_i independently proves highly inefficient. This "iterative" approach requires scanning the text multiple times, resulting in a total time complexity that can be approximately $O(k \cdot (N + M_{\text{avg}}))$ or worse, where M_{avg} is the average pattern length [10]. This highlights the need for more specialized solutions.

2.3.4 The Need for Specialized Multi-Pattern Algorithms

To efficiently handle applications that require simultaneous searches for a large number of patterns, specialized algorithms are essential. These methods usually preprocess the entire set of patterns to create a unified data structure, which enables a single, efficient pass over the text to identify all occurrences of any pattern within the set.

The theoretical framework of *finite automata* provides a robust and elegant solution for this problem. By constructing a single *finite automaton* (FA) that can recognize all patterns in a given set, the text can be processed in a single continuous scan. Each character read from the text triggers a *state transition* within the FA. When the FA reaches an *accepting state*—a state explicitly designed to indicate the successful recognition of one or more patterns—the algorithm reports an occurrence. Algorithms like the Aho-Corasick algorithm exemplify this approach; they automate the construction of such a *pattern-matching machine*, typically based on a trie structure augmented with specialized *failure transitions* [10]. This automaton-based paradigm forms the critical foundation for efficiently solving multi-pattern matching problems.

2.4 THE AHO-CORASICK ALGORITHM

This section provides a comprehensive examination of the Aho-Corasick algorithm, a highly efficient string-matching method designed to locate all occurrences of multiple patterns (keywords) within a given text. It combines *automata theory* with *trie* data structures to efficiently solve the *multi-pattern search* problem. Its search phase runs in $O(n + z)$ time, where n is the text length and z is the number of matches [10].

2.4.1 Context

The task of concurrently identifying all instances of a large collection of patterns within a text poses a significant computational challenge. A naïve approach—running a *single-pattern matching* algorithm such as KMP or BM for each of the k patterns—requires repeated scans of the text and has a worst-case time complexity of $O(k \cdot n)$, where k is the number of patterns and n is the text length, rendering it impractical for large pattern sets or long texts [11].

The Aho-Corasick algorithm overcomes this by building one *finite automaton* that

encodes all patterns simultaneously. This automaton processes the input text in a single pass to detect all matches. The algorithm effectively combines the *state-transition* efficiency of finite automata with the prefix-sharing advantages of tries, following *failure links* on mismatches and resuming matching without rescanning characters—an extension of KMP's *failure-function* idea to a trie of patterns.

2.4.2 Algorithm Components and Construction

Aho-Corasick has two main phases: (i) construction, which builds the automaton, and (ii) search, which scans the text. Three key tables, built in the following order, define the automaton:

Keyword Trie (goto)

A *keyword trie* is first constructed from the pattern set $K = \{P_1, P_2, \ldots, P_k\}$. Each node represents a state, and each edge corresponds to a character transition. A path from the root to any state s spells a prefix of one or more keywords. Formally, the *goto function* goto(s, a) returns the next state s' when an edge labelled a exists; otherwise the transition fails. Undefined transitions from the root are assigned back to the root, i.e., goto$(0, a) = 0$, creating implicit self-loops for every alphabet symbol.

Failure Function (f)

For any state s (except the root), f(s) is the state s' whose path label is the longest proper suffix of the path label of s that is also a prefix of some keyword. On a mismatch, the automaton follows f(s) and resumes matching without consuming the current character. The *failure function* is computed level-by-level (breadth-first) after the goto table is complete.

The Output Function

The *output function*, output(s), identifies all keywords that end at state s. For a given state s, output(s) is the set of all patterns P_i in K such that the string corresponding to the path to s is P_i. To ensure all matches are reported, including those that are suffixes of other patterns, output(s) is augmented during the failure function computation: if f$(s) = s'$, then output$(s) \leftarrow$ output$(s) \cup$ output(f(s)). This incremental union lets each state report every pattern that ends either at that state or at any suffix state reached through its failure chain.

2.4.3 Matching Phase

Once the Aho–Corasick automaton has been built, it scans the text $T = a_1 a_2 \ldots a_n$ in one left-to-right pass. At each position i, the machine attempts to follow the transition labelled a_i from its *current_state* (cs); if that edge exists, it moves along it, otherwise it follows *failure links* (f) until either a matching transition is found or the root is reached. Every time a successful transition is made, the automaton inspects the output set of

the new state and emits all patterns ending at position i. Figure 3 illustrates this control flow in detail.

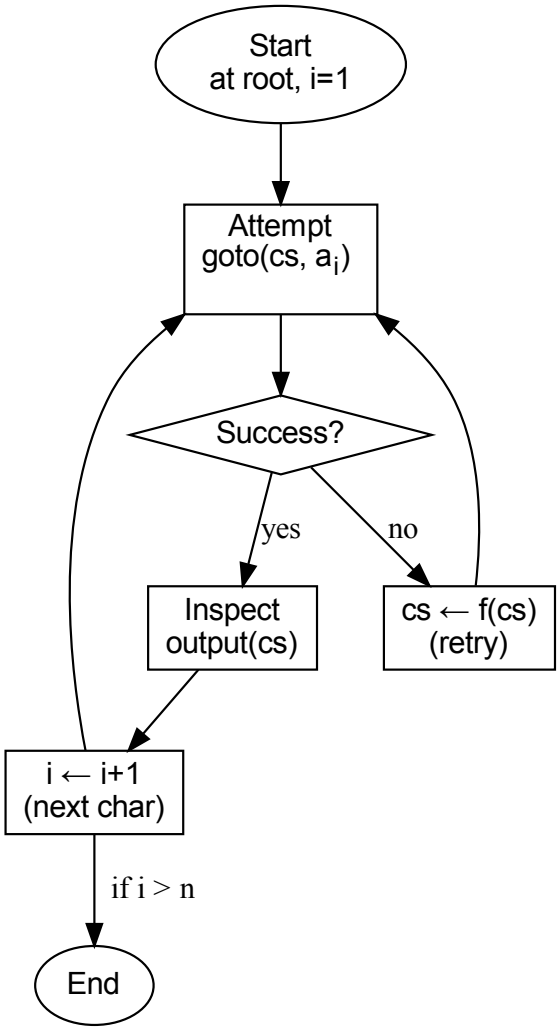

Figure 3: Flowchart of the Aho–Corasick matching phase. Start at the root with $i = 1$, attempt each goto($\textit{current_state}, a_i$), follow failure links on a "no" decision, report matches via output($\textit{current_state}$) on a "yes," advance i, and terminate when $i > n$.

Because each character can trigger at most one eventual successful goto and any number of failure-link traversals, the total number of transitions (goto plus failure) over the entire scan is bounded by $< 2n$ [10].

2.4.4 Time and Space Complexity

The Aho-Corasick algorithm exhibits optimal *linear time complexity* for multi-pattern matching [10, 11].

- **Construction phase**: building the goto, failure, and output functions takes $O(L)$

time, where L is the total length of all keywords. If a dense array is used for the root, an additional $O(|\Sigma|)$ initialization cost is incurred, giving $O(L + |\Sigma|)$.

- **Search phase**: Scanning a text of length n performs fewer than $2n$ state transitions. Reporting matches costs time proportional to the total number of occurrences z, so the total search time is $O(n + z)$.

- **Space complexity**: Storing the automaton requires $O(L + |\Sigma|)$ space: $O(L)$ for trie edges, $O(L)$ pointers for failure links, and additional space for output lists (proportional to the number of stored outputs).

2.4.5 Applications and Relevance

The Aho-Corasick algorithm is widely used in applications that require high-performance *multi-pattern matching*. Its single-pass processing capability makes it ideal for systems such as Network Intrusion Detection Systems (NIDS), which scan network traffic against thousands of malicious signatures [10]. In cybersecurity, it is a core component of anti-malware tools and engines, such as YARA, where it efficiently matches literal string components extracted from complex rules. Other applications include large-scale text analysis, motif searching in DNA or protein sequences, and spam filtering.

While Aho-Corasick excels at exact literal matching, systems like NIDS often layer additional capabilities—such as *regular-expression matching*—on top of Aho-Corasick engines to support more expressive signatures, which affects overall performance characteristics. The algorithm therefore remains a theoretically elegant and practically impactful example of *finite-automata theory* applied to string processing [2, 11].

3

3 SYSTEMATIC REVIEW

This section outlines the methodology employed for the Systematic Literature Review (SLR) conducted to identify, evaluate, and synthesize pertinent studies concerning the parallelization and optimization of the Aho-Corasick algorithm. The review focuses on implementations targeting multi-core CPUs. The objective of this SLR is to support the development of the present research and to answer the formulated research questions.

3.1 RESEARCH QUESTIONS

The following Research Questions (RQs) were formulated to guide the literature search, selection, and analysis process:

1. **RQ1:** What parallelization techniques and associated challenges are documented in the literature for implementing the Aho-Corasick algorithm utilizing threads on multi-core CPUs?

2. **RQ2:** What specific optimizations for the Aho-Corasick algorithm are employed in application scenarios involving extensive pattern sets, particularly within domains of information security?

3.2 SEARCH STRATEGY AND STUDY SELECTION PROTOCOL

The search and selection protocol for identifying primary studies encompassed the definition of data sources, construction of search queries, and establishment of inclusion and exclusion criteria.

3.2.1 Data Sources

The following scientific databases were selected due to their scope and relevance in the area of Computer Science and Software Engineering:

- Web of Science (WoS)
- Scopus
- ACM Digital Library (ACM DL)

- IEEE Xplore

3.2.2 Search Queries

Two primary search strings were constructed to address the aspects of parallelization (RQ1) and optimization/application (RQ2) pertinent to the Aho-Corasick algorithm. The literature search was executed in May 2025.

String 1 (Focus on Parallelization):

```
"Aho-Corasick" AND (parallel* OR multithread* OR multi-thread* OR
thread* OR concurren* OR hpc OR high-perform* OR high perform*) AND
("multi-core" OR multicore OR "many-core" OR CPU) NOT "distributed
systems")
```

String 2 (Focus on Optimization and Applications):

```
"Aho-Corasick" AND (optimi* OR accelerat* OR perform* OR evaluat* OR
improve*) AND ("application" OR "implementation" OR "use case"))
```

3.2.3 Inclusion (IC) and Exclusion (EC) Criteria

The subsequent criteria were established for the selection of primary studies:

Inclusion Criteria (IC):

- **IC1 (Primary Focus):** The study addresses the Aho-Corasick (AC) algorithm or presents parallelization/optimization techniques demonstrably applicable to AC.
- **IC2 (Parallelism Aspect):** The study investigates, proposes, implements, analyzes, or explicitly discusses the parallelization or concurrent execution of the Aho-Corasick algorithm.
- **IC3 (Target Platform):** The study reports empirical results or analyses about multi-core CPU platforms.
- **IC4 (RQ Relevance):** The study provides information relevant to addressing at least one RQ.
- **IC6 (Language):** Published in English or Portuguese.
- **IC7 (Accessibility):** Full-text availability of the study.
- **IC8 (Publication Window):** Published between January 2015 and May 2025, inclusive.

Exclusion Criteria (EC):

- **EC1 (Irrelevant Focus):** Studies with only superficial mentions of the Aho-Corasick algorithm or pattern matching, lacking a primary focus on its parallelization or optimization.

- **EC2 (Non-CPU Platform Exclusivity):** Studies exclusively focused on non-CPU platforms (e.g., GPU, FPGA) without analysis or insights transferable to multi-core CPU environments.
- **EC4 (Insufficient Detail):** Studies lacking sufficient detail regarding methodology, implementation, or reported results.
- **EC5 (Publication Type):** Opinion pieces, editorials, and non-peer-reviewed gray literature.
- **EC6 (Language Barrier):** Publications in languages other than English or Portuguese.
- **EC7 (Duplication):** Duplicate publications (the most comprehensive or recent version was retained).
- **EC8 (Inaccessibility):** Full-text unobtainable.

3.2.4 Selection Process and Initial Search Outcomes

The study selection was a multi-stage process. Initially, the constructed search strings were executed against the selected digital libraries. The retrieved results were subsequently refined using filters provided by each database (e.g., publication period, document type, language, subject area), as detailed below:

- **ACM Digital Library:** String 1 yielded 122 articles, reduced to 29 following filter application
  String 2 yielded 274 articles, reduced to 66 following filter application
  Filters applied: Publication years: 2015-2025; Document type: Research Article; Publisher: ACM; Title exclusion: GPU; Language: English.
- **Web of Science:** String 1 yielded 30 articles, reduced to 6 after filter application
  String 2 yielded 66 articles, reduced to 20 after filter application.
  Filters applied: Publication years: 2015-2025; WoS Categories: Computer Science (excluding Artificial Intelligence); Title exclusion: GPU; Language: English.
- **Scopus:** String 1 yielded 34 articles, reduced to 6 following filter application
  String 2 yielded 145 articles, reduced to 49 following filter application
  Filters applied: Publication years: 2015-2025 Language: English Subject areas: Computer Science, Engineering; Document types: Article, Conference Paper; Title exclusion: GPU.
- **IEEE Xplore:** String 1 yielded 25 articles, reduced to 2 following filter application
  String 2 yielded 60 articles, reduced to 18 following filter application

Filters applied: Publication years: 2015-2025; Title exclusion: GPU; Language: English.

Following the initial database search and filtering stage, the resultant sets of candidate articles were aggregated.

The filtered outputs from each database and search query were subsequently merged, and duplicate entries were eliminated. Thereafter, a title and abstract screening was performed on the unique articles, and the established inclusion and exclusion criteria were applied.

4

4 PROPOSAL

Therefore, this research proposes a *data-parallel scanning strategy* for YARA's Aho-Corasick engine on multi-core CPUs. The proposed solution involves partitioning each large input file into fixed-size, overlapping chunks and using multiple `POSIX threads` to concurrently traverse the same pre-compiled Aho-Corasick automaton. Each thread will process its assigned chunk, and thread-local match lists will be merged upon scan completion. The originality of this work lies not in inventing a new parallel algorithm, but in the software engineering required to integrate this strategy into YARA's complex `C`-based codebase and the scientific rigor of its empirical evaluation.

4.1 CRONOGRAMA

This schedule details the activities carried out and planned for the development of the present work, distributed throughout the academic year. The tables below present the main phases of the project, from the conception and initial research to the implementation, evaluation and final writing of the document, divided into two semesters for better visualization.

Table 1: Cronograma de Atividades do TCC (Fev-Jul)

Atividade	**Fev**	**Mar**	**Abr**	**Mai**	**Jun**	**Jul**
Fase 1: Pesquisa e Planejamento						
Orientação e Definição do Tema	X	X				
Revisão Sistemática da Literatura	X	X	X	X		
Estudo da Fundamentação Teórica		X	X			
Desenvolvimento da Proposta de Pesquisa			X	X		
Fase 2: Escrita e Submissão Parcial						
Escrita dos Capítulos Iniciais				X	X	
Submissão do Projeto e Documento Parcial					X	
Fase 3: Implementação e Desenvolvimento						
Análise do Código-Fonte do YARA					X	
Desenvolvimento do Protótipo Paralelo						
Resolução de Desafios de Integração						

Table 2: Cronograma de Atividades do TCC (Ago-Dez)

Atividade	**Ago**	**Set**	**Out**	**Nov**	**Dez**
Fase 3: Implementação e Desenvolvimento					
Análise do Código-Fonte do YARA	X				
Desenvolvimento do Protótipo Paralelo	X	X			
Resolução de Desafios de Integração	X	X			
Fase 4: Avaliação Experimental					
Definição do Ambiente e Casos de Teste	X	X			
Execução dos Experimentos de Desempenho		X	X		
Coleta e Análise dos Resultados			X		
Fase 5: Finalização e Defesa					
Escrita dos Capítulos Finais				X	X
Revisão e Formatação Final do TCC					X
Preparação para a Apresentação Final					X
Submissão Final e Defesa do TCC					X

REFERENCES

References

[1] P. Pacheco, *An Introduction to Parallel Programming*, 2nd ed. Morgan Kaufmann, 2011.

[2] VirusTotal, "YARA: The Pattern Matching Swiss Knife," https://virustotal.github.io/yara/, 2025, [Online; accessed 8 May 2025].

[3] V. Thambawita, R. G. Ragel, and D. Elkaduwe, "An optimized parallel failure-less aho-corasick algorithm for DNA sequence matching," *CoRR*, vol. abs/1811.10498, 2018.

[4] D. R. Butenhof, *Programming with POSIX Threads*. Addison-Wesley, 1997.

[5] M. J. Flynn, "Very high-speed computing systems," *Proceedings of the IEEE*, vol. 54, no. 12, pp. 1901–1909, 1966.

[6] A. Grama, A. Gupta, G. Karypis, and V. Kumar, *Introduction to Parallel Computing*, 2nd ed. Addison-Wesley, 2003.

[7] E. Fredkin, "Trie memory," *Communications of the ACM*, vol. 3, no. 9, pp. 490–499, 1960.

[8] D. E. Knuth, *The Art of Computer Programming, Volume 3: Sorting and Searching*, 2nd ed. Addison-Wesley, 1998.

[9] R. Sedgewick and K. Wayne, *Algorithms*, 4th ed. Addison-Wesley Professional, 2011.

[10] A. V. Aho and M. J. Corasick, "Efficient string matching: An aid to bibliographic search," *Communications of the ACM*, vol. 18, no. 6, pp. 333–343, 1975.

[11] D. Gusfield, *Algorithms on Strings, Trees, and Sequences: Computer Science and Computational Biology*. Cambridge University Press, 1997.

[12] D. E. Knuth, J. H. Morris Jr., and V. R. Pratt, "Fast pattern matching in strings," *SIAM Journal on Computing*, vol. 6, no. 2, pp. 323–350, 1977.

[13] R. S. Boyer and J. S. Moore, "A fast string searching algorithm," *Communications of the ACM*, vol. 20, no. 10, pp. 762–772, 1977.

Ensino que
te **conecta**